VIT-AP UNIVERSITY

LAB SHEET - 6

NAME – Sk . Waseem Akram

REG No – 23BCE7706

Faculty – Prof . S . Gopikrishnan

School – Scope

Date – 11 - 03 - 26

1. Use MongoDB to implement the following DB operations

   1. Create a database called ‘vehicles’ and *write* a MongoDB query to select database as “vehicles”.

```
> use vehicles
< switched to db vehicles
vehicles>
```

   2. Write a MongoDB query to display all the databases.

```
> show dbs
< admin   40.00 KiB
  config  12.00 KiB
  local   40.00 KiB
vehicles>
```

   3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

```
> db.createCollection("four_wheelers")
< { ok: 1 }
```

```
> db.createCollection("two_wheelers", { capped: true, size: 5242880, max: 5000 })
< { ok: 1 }
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
> db.two_wheelers.insertMany([
    { bike_name: "Activa 6G", model: "gearless", category: "110cc", colors_available: ["red", "black", "white"], manufacturer: "Honda"
    { bike_name: "Pulsar 150", model: "gear", category: "150cc", colors_available: ["blue", "black", "red"], manufacturer: "Bajaj", pe
    { bike_name: "Duke 200", model: "gear", category: "200cc", colors_available: ["orange", "white"], manufacturer: "KTM", performance
    { bike_name: "Access 125", model: "gearless", category: "125cc", colors_available: ["grey", "silver"], manufacturer: "Suzuki", per
    { bike_name: "Splendor Plus", model: "gear", category: "100cc", colors_available: ["black", "purple"], manufacturer: "Hero", perfo
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b17dfc5980d272c3844d11'),
      '1': ObjectId('69b17dfc5980d272c3844d12'),
      '2': ObjectId('69b17dfc5980d272c3844d13'),
      '3': ObjectId('69b17dfc5980d272c3844d14'),
      '4': ObjectId('69b17dfc5980d272c3844d15')
    }
  }
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
> db.four_wheelers.insertMany([
    { vehicle_name: "Swift", model: "own", category: "car", variants: ["vxi", "zxi", "petrol"], manufacturer: "Maruti", performance: 8
    { vehicle_name: "Eicher Pro", model: "commercial", category: "lorry", variants: ["diesel", "cng"], manufacturer: "Eicher", perform
    { vehicle_name: "City", model: "own", category: "car", variants: ["vxi", "petrol", "hybrid"], manufacturer: "Honda", performance:
    { vehicle_name: "Traveller", model: "commercial", category: "bus", variants: ["diesel"], manufacturer: "Force", performance: 6, ti
    { vehicle_name: "Ace", model: "commercial", category: "mini truck", variants: ["diesel", "petrol"], manufacturer: "Tata", performa
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b17e2d5980d272c3844d16'),
      '1': ObjectId('69b17e2d5980d272c3844d17'),
      '2': ObjectId('69b17e2d5980d272c3844d18'),
      '3': ObjectId('69b17e2d5980d272c3844d19'),
      '4': ObjectId('69b17e2d5980d272c3844d1a')
    }
  }
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
> db.two_wheelers.find().pretty()
< {
    _id: ObjectId('69b17dfc5980d272c3844d11'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '110cc',
    colors_available: [
      'red',
      'black',
      'white'
    ],
    manufacturer: 'Honda',
    performance: 7,
    timestamp: 2020-01-15T00:00:00.000Z,
    price: 75000
  }
  {
    _id: ObjectId('69b17dfc5980d272c3844d12'),
    bike_name: 'Pulsar 150',
    model: 'gear',
    category: '150cc',
    colors_available: [
      'blue',
      'black',
      'red'
    ],
    manufacturer: 'Bajaj',
    performance: 8,
    timestamp: 2021-05-20T00:00:00.000Z,
```

```
}
{
  _id: ObjectId('69b17dfc5980d272c3844d14'),
  bike_name: 'Access 125',
  model: 'gearless',
  category: '125cc',
  colors_available: [
    'grey',
    'silver'
  ],
  manufacturer: 'Suzuki',
  performance: 8,
  timestamp: 2019-11-25T00:00:00.000Z,
  price: 82000
}
{
  _id: ObjectId('69b17dfc5980d272c3844d15'),
  bike_name: 'Splendor Plus',
  model: 'gear',
  category: '100cc',
  colors_available: [
    'black',
    'purple'
  ],
  manufacturer: 'Hero',
  performance: 6,
  timestamp: 2023-01-05T00:00:00.000Z,
  price: 72000
}
```

```
> db.four_wheelers.find().pretty()
< {
    _id: ObjectId('69b17e2d5980d272c3844d16'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [
      'vxi',
      'zxi',
      'petrol'
    ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: 2021-06-12T00:00:00.000Z,
    price: 700000
  }
  {
    _id: ObjectId('69b17e2d5980d272c3844d17'),
    vehicle_name: 'Eicher Pro',
    model: 'commercial',
    category: 'lorry',
    variants: [
      'diesel',
      'cng'
    ],
    manufacturer: 'Eicher',
    performance: 7,
    timestamp: 2020-08-15T00:00:00.000Z,
    price: 1500000
```

```
{
  _id: ObjectId('69b17e2d5980d272c3844d18'),
  vehicle_name: 'City',
  model: 'own',
  category: 'car',
  variants: [
    'vxi',
    'petrol',
    'hybrid'
  ],
  manufacturer: 'Honda',
  performance: 9,
  timestamp: 2022-10-01T00:00:00.000Z,
  price: 1200000
}
{
  _id: ObjectId('69b17e2d5980d272c3844d19'),
  vehicle_name: 'Traveller',
  model: 'commercial',
  category: 'bus',
  variants: [
    'diesel'
  ],
  manufacturer: 'Force',
  performance: 6,
  timestamp: 2018-04-22T00:00:00.000Z,
  price: 1800000
}
{
```

```
}
{
  _id: ObjectId('69b17e2d5980d272c3844d1a'),
  vehicle_name: 'Ace',
  model: 'commercial',
  category: 'mini truck',
  variants: [
    'diesel',
    'petrol'
  ],
  manufacturer: 'Tata',
  performance: 7,
  timestamp: 2023-02-14T00:00:00.000Z,
  price: 550000
}
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
> db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
< {
    bike_name: 'Activa 6G',
    price: 75000
  }
  {
    bike_name: 'Pulsar 150',
    price: 110000
  }
  {
    bike_name: 'Duke 200',
    price: 190000
  }
  {
    bike_name: 'Access 125',
    price: 82000
  }
  {
    bike_name: 'Splendor Plus',
    price: 72000
  }
```

```
> db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })
< {
    vehicle_name: 'Swift',
    price: 700000
  }
  {
    vehicle_name: 'Eicher Pro',
    price: 1500000
  }
  {
    vehicle_name: 'City',
    price: 1200000
  }
  {
    vehicle_name: 'Traveller',
    price: 1800000
  }
  {
    vehicle_name: 'Ace',
    price: 550000
  }
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
> db.two_wheelers.find({ manufacturer: "Honda" })
< {
    _id: ObjectId('69b17dfc5980d272c3844d11'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '110cc',
    colors_available: [
      'red',
      'black',
      'white'
    ],
    manufacturer: 'Honda',
    performance: 7,
    timestamp: 2020-01-15T00:00:00.000Z,
    price: 75000
  }
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
> db.four_wheelers.find({ variants: "diesel" })
< {
    _id: ObjectId('69b17e2d5980d272c3844d17'),
    vehicle_name: 'Eicher Pro',
    model: 'commercial',
    category: 'lorry',
    variants: [
      'diesel',
      'cng'
    ],
    manufacturer: 'Eicher',
    performance: 7,
    timestamp: 2020-08-15T00:00:00.000Z,
    price: 1500000
  }
  {
    _id: ObjectId('69b17e2d5980d272c3844d19'),
    vehicle_name: 'Traveller',
    model: 'commercial',
    category: 'bus',
    variants: [
      'diesel'
    ],
    manufacturer: 'Force',
    performance: 6,
    timestamp: 2018-04-22T00:00:00.000Z,
    price: 1800000
  }
  {
    _id: ObjectId('69b17e2d5980d272c3844d1a'),
```

```
}
{
  _id: ObjectId('69b17e2d5980d272c3844d1a'),
  vehicle_name: 'Ace',
  model: 'commercial',
  category: 'mini truck',
  variants: [
    'diesel',
    'petrol'
  ],
  manufacturer: 'Tata',
  performance: 7,
  timestamp: 2023-02-14T00:00:00.000Z,
  price: 550000
}
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
> db.two_wheelers.find({ performance: { $gt: 5 } }, { bike_name: 1, category: 1, manufacturer: 1, _id: 0 })
< {
    bike_name: 'Activa 6G',
    category: '110cc',
    manufacturer: 'Honda'
  }
  {
    bike_name: 'Pulsar 150',
    category: '150cc',
    manufacturer: 'Bajaj'
  }
  {
    bike_name: 'Duke 200',
    category: '200cc',
    manufacturer: 'KTM'
  }
  {
    bike_name: 'Access 125',
    category: '125cc',
    manufacturer: 'Suzuki'
  }
  {
    bike_name: 'Splendor Plus',
    category: '100cc',
    manufacturer: 'Hero'
  }
```

```
> db.four_wheelers.find({ performance: { $gt: 5 } }, { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 })
< {
    vehicle_name: 'Swift',
    category: 'car',
    manufacturer: 'Maruti'
  }
  {
    vehicle_name: 'Eicher Pro',
    category: 'lorry',
    manufacturer: 'Eicher'
  }
  {
    vehicle_name: 'City',
    category: 'car',
    manufacturer: 'Honda'
  }
  {
    vehicle_name: 'Traveller',
    category: 'bus',
    manufacturer: 'Force'
  }
  {
    vehicle_name: 'Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  }
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and *write* a MongoDB query to select database as ‘animal’.

```
> use animal
< switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
> show dbs
< admin      40.00 KiB
  config     92.00 KiB
  local      40.00 KiB
  vehicles   80.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
> db.createCollection("wild_animals", { capped: true, size: 5242880, max: 5000 })
< { ok: 1 }
> db.createCollection("domestic_animals")
< { ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
> db.wild_animals.insertMany([
    { animal_name: "Lion", nature: "harm", favorite_foods: ["meat", "deer"], care_taker_name: "Arthur", life_span: 14, timestamp: new Dat
    { animal_name: "Tiger", nature: "harm", favorite_foods: ["meat", "boar"], care_taker_name: "Arthur", life_span: 15, timestamp: new Da
    { animal_name: "Elephant", nature: "harmless", favorite_foods: ["grass", "sugarcane"], care_taker_name: "Vikram", life_span: 60, time
    { animal_name: "Bear", nature: "harm", favorite_foods: ["fish", "honey"], care_taker_name: "Sarah", life_span: 25, timestamp: new Dat
    { animal_name: "Giraffe", nature: "harmless", favorite_foods: ["acacia leaves"], care_taker_name: "Vikram", life_span: 25, timestamp:
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b18052d44d597841486b4b'),
      '1': ObjectId('69b18052d44d597841486b4c'),
      '2': ObjectId('69b18052d44d597841486b4d'),
      '3': ObjectId('69b18052d44d597841486b4e'),
      '4': ObjectId('69b18052d44d597841486b4f')
    }
  }
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
> db.domestic_animals.insertMany([
    { animal_name: "Dog", gender: "male", favorite_foods: ["meat", "dog food"], animal_petname: "Buddy", life_span: 12, timestamp: new Da
    { animal_name: "Cat", gender: "female", favorite_foods: ["fish", "milk"], animal_petname: "Luna", life_span: 14, timestamp: new Date(
    { animal_name: "Horse", gender: "male", favorite_foods: ["grass", "hay"], animal_petname: "Chetak", life_span: 25, timestamp: new Dat
    { animal_name: "Rabbit", gender: "female", favorite_foods: ["carrots", "lettuce"], animal_petname: "Snowball", life_span: 8, timestam
    { animal_name: "Cow", gender: "female", favorite_foods: ["grass", "hay"], animal_petname: "Gauri", life_span: 20, timestamp: new Date
  ])

< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b18068d44d597841486b50'),
      '1': ObjectId('69b18068d44d597841486b51'),
      '2': ObjectId('69b18068d44d597841486b52'),
      '3': ObjectId('69b18068d44d597841486b53'),
      '4': ObjectId('69b18068d44d597841486b54')
    }
  }
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
> db.wild_animals.find()
< {
    _id: ObjectId('69b18052d44d597841486b4b'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'deer'
    ],
    care_taker_name: 'Arthur',
    life_span: 14,
    timestamp: 2021-03-15T00:00:00.000Z,
    expenses: 50000
  }
  {
    _id: ObjectId('69b18052d44d597841486b4c'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'boar'
    ],
    care_taker_name: 'Arthur',
    life_span: 15,
    timestamp: 2020-07-22T00:00:00.000Z,
    expenses: 45000
  }
  {
    _id: ObjectId('69b18052d44d597841486b4d'),
    animal_name: 'Elephant',
```

```
  care_taker_name: 'Vikram',
  life_span: 60,
  timestamp: 2015-01-10T00:00:00.000Z,
  expenses: 30000
}
{
  _id: ObjectId('69b18052d44d597841486b4e'),
  animal_name: 'Bear',
  nature: 'harm',
  favorite_foods: [
    'fish',
    'honey'
  ],
  care_taker_name: 'Sarah',
  life_span: 25,
  timestamp: 2019-11-05T00:00:00.000Z,
  expenses: 20000
}
{
  _id: ObjectId('69b18052d44d597841486b4f'),
  animal_name: 'Giraffe',
  nature: 'harmless',
  favorite_foods: [
    'acacia leaves'
  ],
  care_taker_name: 'Vikram',
  life_span: 25,
  timestamp: 2018-09-30T00:00:00.000Z,
  expenses: 15000
}
```

```
> db.domestic_animals.find()
< {
    _id: ObjectId('69b18068d44d597841486b50'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [
      'meat',
      'dog food'
    ],
    animal_petname: 'Buddy',
    life_span: 12,
    timestamp: 2022-05-10T00:00:00.000Z,
    expenses: 5000
  }
  {
    _id: ObjectId('69b18068d44d597841486b51'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [
      'fish',
      'milk'
    ],
    animal_petname: 'Luna',
    life_span: 14,
    timestamp: 2023-01-20T00:00:00.000Z,
    expenses: 3000
  }
  {
    _id: ObjectId('69b18068d44d597841486b52'),
    animal_name: 'Horse',
    gender: 'male',
```

```
  animal_petname: 'Chetak',
  life_span: 25,
  timestamp: 2017-08-14T00:00:00.000Z,
  expenses: 15000
}
{
  _id: ObjectId('69b18068d44d597841486b53'),
  animal_name: 'Rabbit',
  gender: 'female',
  favorite_foods: [
    'carrots',
    'lettuce'
  ],
  animal_petname: 'Snowball',
  life_span: 8,
  timestamp: 2023-11-01T00:00:00.000Z,
  expenses: 1000
}
{
  _id: ObjectId('69b18068d44d597841486b54'),
  animal_name: 'Cow',
  gender: 'female',
  favorite_foods: [
    'grass',
    'hay'
  ],
  animal_petname: 'Gauri',
  life_span: 20,
  timestamp: 2019-02-28T00:00:00.000Z,
  expenses: 8000
}
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
> db.domestic_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
< {
    animal_name: 'Dog',
    expenses: 5000
  }
  {
    animal_name: 'Cat',
    expenses: 3000
  }
  {
    animal_name: 'Horse',
    expenses: 15000
  }
  {
    animal_name: 'Rabbit',
    expenses: 1000
  }
  {
    animal_name: 'Cow',
    expenses: 8000
  }
```

```
> db.wild_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
< {
    animal_name: 'Lion',
    expenses: 50000
  }
  {
    animal_name: 'Tiger',
    expenses: 45000
  }
  {
    animal_name: 'Elephant',
    expenses: 30000
  }
  {
    animal_name: 'Bear',
    expenses: 20000
  }
  {
    animal_name: 'Giraffe',
    expenses: 15000
  }
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
> db.domestic_animals.find({ life_span: 12 })
< {
    _id: ObjectId('69b18068d44d597841486b50'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [
      'meat',
      'dog food'
    ],
    animal_petname: 'Buddy',
    life_span: 12,
    timestamp: 2022-05-10T00:00:00.000Z,
    expenses: 5000
  }
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
> db.wild_animals.find({ care_taker_name: "Arthur" })
< {
    _id: ObjectId('69b18052d44d597841486b4b'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'deer'
    ],
    care_taker_name: 'Arthur',
    life_span: 14,
    timestamp: 2021-03-15T00:00:00.000Z,
    expenses: 50000
  }
  {
    _id: ObjectId('69b18052d44d597841486b4c'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'boar'
    ],
    care_taker_name: 'Arthur',
    life_span: 15,
    timestamp: 2020-07-22T00:00:00.000Z,
    expenses: 45000
  }
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
    favorite_foods: [
      'fish',
      'honey'
    ],
    expenses: 20000
  }
  {
    animal_name: 'Giraffe',
    favorite_foods: [
      'acacia leaves'
    ],
    expenses: 15000
  }
> db.domestic_animals.find({ life_span: { $gt: 5 } }, { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 })
< {
    animal_name: 'Dog',
    favorite_foods: [
      'meat',
      'dog food'
    ],
    expenses: 5000
  }
  {
    animal_name: 'Cat',
    favorite_foods: [
      'fish',
      'milk'
    ],
    expenses: 3000
  }
```

```
> db.wild_animals.find({ life_span: { $gt: 5 } }, { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 })
< {
    animal_name: 'Lion',
    favorite_foods: [
      'meat',
      'deer'
    ],
    expenses: 50000
  }
  {
    animal_name: 'Tiger',
    favorite_foods: [
      'meat',
      'boar'
    ],
    expenses: 45000
  }
  {
    animal_name: 'Elephant',
    favorite_foods: [
      'grass',
      'sugarcane'
    ],
    expenses: 30000
  }
  {
    animal_name: 'Bear',
    favorite_foods: [
      'fish',
      'honey'
```

```
    'fish',
    'milk'
  ],
  expenses: 3000
}
{
  animal_name: 'Horse',
  favorite_foods: [
    'grass',
    'hay'
  ],
  expenses: 15000
}
{
  animal_name: 'Rabbit',
  favorite_foods: [
    'carrots',
    'lettuce'
  ],
  expenses: 1000
}
{
  animal_name: 'Cow',
  favorite_foods: [
    'grass',
    'hay'
  ],
  expenses: 8000
}
```